

COMP 4211 Notes

Supervised learning: Given $\mathcal{S} = \{(\mathbf{x}^{(l)}, \mathbf{y}^{(l)})\}_{l=1}^N$. Find f using $\mathcal{S}$ such that the predicted output $f(\mathbf{x}^{(l)})$ is close to the actual output $\mathbf{y}^{(l)}$.

Classification task: Output is categorical.

Regression task: Output is continuous-valued.

Unsupervised learning: Given $\mathcal{S} = \{\mathbf{x}^{(l)}\}_{l=1}^N$ to perform some tasks.

- Linear Regression

In general, given $\mathcal{S} = \{(\mathbf{x}^{(l)}, \mathbf{y}^{(l)})\}_{l=1}^N$, we want $f(\cdot; \mathbf{w})$.

In linear regression, the regression function is a linear function $f(\mathbf{x}; \mathbf{w}) = w_0 + w_1x_1 + \dots + w_dx_d = \mathbf{w}^T \tilde{\mathbf{x}}$,

$$\text{where } \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} \text{ and } \tilde{\mathbf{x}} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}.$$

We define a loss function $L(\mathbf{w}; \mathcal{S})$. The most common loss function for regression problems is the squared loss.

$$L(\mathbf{w}; \mathcal{S}) = \sum_{l=1}^N (f(\mathbf{x}^{(l)}; \mathbf{w}) - y^{(l)})^2 = \sum_{l=1}^N (w_0 + w_1x_1^{(l)} + \dots + w_dx_d^{(l)} - y^{(l)})^2$$

$$\text{Let } \tilde{\mathbf{X}} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{bmatrix}. \text{ By mathematics, we have } \hat{\mathbf{w}} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \mathbf{y}.$$

The squared loss can also be expressed as $L(\mathbf{w}; \mathcal{S}) = \|\tilde{\mathbf{X}}\mathbf{w} - \mathbf{y}\|^2$.

When d is large, we can estimate $\hat{\mathbf{w}}$ iteratively.

- Nonlinear Extension

Instead of $f(\mathbf{x})$, let the function be

1. $f(\mathbf{x}, \varphi_1(\mathbf{x}), \dots, \varphi_k(\mathbf{x}); \mathbf{w})$, where f linear, $\varphi_1, \dots, \varphi_k$ nonlinear.
2. $\varphi(\mathbf{x})$, where φ nonlinear.
3. $f(\Phi(\mathbf{x}); \mathbf{w})$, where f linear, the nonlinear mapping $\Phi: \mathbb{R}^d \rightarrow \mathbb{R}^k, k \gg d$ defined implicitly.

Method 1 leads to polynomial regression: $f(x; \mathbf{w}) = w_0 + w_1x + \dots + w_mx^m$.

- Regularization

Sometimes overfitting occurs. Hence we define our regularized loss function

$$L_\lambda(\mathbf{w}; \mathcal{S}) = L(\mathbf{w}; \mathcal{S}) + \lambda \|\mathbf{w}\|^2 = \|\tilde{\mathbf{X}}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2, \text{ where } \lambda > 0 \text{ is called the regularization parameter.}$$

In practice, not regularizing w_0 usually gives better result.

In this case, we have $\hat{\mathbf{w}} = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} + \lambda \mathbf{I})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$

We can use cross validation to select the optimized value of the hyper-parameters like λ .

Other regularizations: Using L_p norm in $\|\mathbf{w}\|$ for general $p \geq 0$ instead of L_2 norm. In particular, L_1 regularization is called LASSO, favors sparse solutions and has no closed-form solution.

Metric:

$$\text{Mean square error MSE} = \frac{1}{N} \sum_{l=1}^n (f(\mathbf{x}^{(l)}; \mathbf{w}) - y^{(l)})^2$$

We can also use root mean square error $\text{RMSE} = \sqrt{\text{MSE}}$

$$\text{We can also use the coefficient of determination } R^2 = 1 - \frac{\sum_{l=1}^n (f(\mathbf{x}^{(l)}; \mathbf{w}) - y^{(l)})^2}{\sum_{l=1}^N (\bar{y} - y^{(l)})^2} = 1 - \frac{\text{MSE}}{\text{Var}(y)}$$

- Logistic Regression

Binary classification: Training set $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^N$, where $y \in \{0, 1\}$.

Multiclass classification: Training set $\mathcal{S} = \{(\mathbf{x}^{(l)}, \mathbf{y}^{(l)})\}_{l=1}^N$, where $\mathbf{y}^{(l)} \in \{0, 1\}^K$ uses one-hot encoding such that $y_i^{(l)} = 0 \forall i \neq j$, $y_j^{(l)} = 1$ if $\mathbf{x}^{(l)}$ belongs to class j .

- Binary Classification

$$\text{Logistic function } \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

$$\text{General logistic function } \sigma_a(z) = \frac{1}{1 + e^{-az}} \in (0, 1). \text{ When } a \rightarrow \infty, \sigma_a(z) \text{ approaches the step function.}$$

Logistic regression model: $f(\mathbf{x}; \mathbf{w}) = \sigma(\mathbf{w}^\top \tilde{\mathbf{x}})$

$\prod_{l=1}^N (f(\mathbf{x}^{(l)}; \mathbf{w}))^{y^{(l)}} (1 - f(\mathbf{x}^{(l)}; \mathbf{w}))^{1-y^{(l)}}$ is called the likelihood, which is what we want to maximize w.r.t. $\mathbf{w}$.

$$\text{Taking } -\log \text{ to the likelihood, we get } L(\mathbf{w}; \mathcal{S}) = - \sum_{l=1}^N (y^{(l)} \log f(\mathbf{x}^{(l)}; \mathbf{w}) + (1 - y^{(l)}) \log(1 - f(\mathbf{x}^{(l)}; \mathbf{w})))$$

$L(\mathbf{w}; \mathcal{S})$ is convex, but no-closed form solution exists. We use gradient descent.

A nice property about $\sigma(z)$: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

$$\text{By mathematics, we have } \frac{\partial L}{\partial w_i} = - \sum_{l=1}^N (y^{(l)} - f(\mathbf{x}^{(l)}; \mathbf{w})) x_i^{(l)}, \nabla_{\mathbf{w}} L = - \sum_{l=1}^N (y^{(l)} - f(\mathbf{x}^{(l)}; \mathbf{w})) \tilde{\mathbf{x}}^{(l)}$$

$$\text{Hence } \Delta w_i = -\eta \frac{\partial L}{\partial w_i} = \eta \sum_{l=1}^N (y^{(l)} - f(\mathbf{x}^{(l)}; \mathbf{w})) x_i^{(l)}, \Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L = \eta \sum_{l=1}^N (y^{(l)} - f(\mathbf{x}^{(l)}; \mathbf{w})) \tilde{\mathbf{x}}^{(l)}$$

- Multiclass classification

$$\text{Softmax function } \text{softmax}(\mathbf{z}) = \mathbf{r}, \text{ where } r_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \in (0, 1).$$

$$\frac{\partial r_j}{\partial z_k} = r_j(\delta_{jk} - r_k) = \begin{cases} r_j(1 - r_j) & j = k \\ -r_j r_k & j \neq k \end{cases}$$

Let $\mathbf{W} = [w_{ki}] \in \mathbb{R}^{K \times (d+1)}$ be the parameter matrix.

The model becomes: $f(\mathbf{x}; \mathbf{W}) = \text{softmax}(\mathbf{W}\tilde{\mathbf{x}}) \in (0,1)^K$

$$\text{Likelihood: } \prod_{l=1}^N \prod_{j=1}^K \left(r_j^{(l)}\right)^{y_j^{(l)}} = \prod_{l=1}^N r_{\arg_j(y_j^{(l)}=1)}^{(l)}$$

$$\text{Cross-entropy loss: } L(\mathbf{w}; \mathcal{S}) = - \sum_{l=1}^N \sum_{k=1}^K y_j^{(l)} \log r_j^{(l)} = - \sum_{l=1}^N \log r_{\arg_j(y_j^{(l)}=1)}^{(l)}$$

$$\text{By mathematics, we have } \frac{\partial L}{\partial w_{ki}} = - \sum_{l=1}^N \left(y_k^{(l)} - r_k^{(l)}\right) x_i^{(l)}, \Delta w_{ki} = -\eta \frac{\partial L}{\partial w_{ki}} = \eta \sum_{l=1}^N \left(y_k^{(l)} - r_k^{(l)}\right) x_i^{(l)}$$

$$\Delta \mathbf{w}_k^\top = \eta \sum_{l=1}^N \left(y_k^{(l)} - r_k^{(l)}\right) \tilde{\mathbf{x}}^{(l)}$$

We regularize on $\|\mathbf{W}\|_F^2$. Not regularizing the w_{k0} 's may give a better result.

	Pred: Positive	Pred: Negative
Actual: Positive	TP	FN
Actual: Negative	FP	TN

$$\text{Precision: } P = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall: } R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Accuracy: } \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{F1 score: } F_1 = \frac{2PR}{P + R}$$

- Feedforward Neural Networks (MLP)

$$\mathbf{x}^{[1]} = \mathbf{W}^{[1]}\mathbf{x}^{[0]}, \mathbf{z}^{[1]} = f^{[1]}(\mathbf{x}^{[1]}), \mathbf{x}^{[2]} = \mathbf{W}^{[2]}\mathbf{z}^{[1]}, \mathbf{z}^{[2]} = f^{[2]}(\mathbf{x}^{[2]}), \mathbf{x}^{[3]} = \mathbf{W}^{[3]}\mathbf{z}^{[2]}, \mathbf{z}^{[3]} = f^{[3]}(\mathbf{x}^{[3]})$$

$$\text{In general, } \mathbf{z}^{[k]} = f^{[k]}(\mathbf{W}^{[k]}\mathbf{z}^{[k-1]})$$

$$\text{Squared loss function for regression problems: } L(\mathbf{W}; \mathcal{S}) = \frac{1}{2} \sum_{l=1}^N \sum_i \left(z_i^{[m](l)} - y_i^{(l)}\right)^2$$

$$\text{Cross-entropy loss function for classification problems: } L(\mathbf{W}; \mathcal{S}) = - \sum_{l=1}^N \sum_i y_i^{(l)} \log z_i^{[m](l)}$$

$$\text{Weight update rule for the last layer: } \begin{cases} \delta_j^{[m](l)} = y_j^{(l)} - z_j^{[m](l)} \\ \Delta w_{ji}^{[m]} = -\eta \frac{\partial L}{\partial w_{ji}^{[m]}} = \eta \sum_{l=1}^N \delta_j^{[m](q)} z_i^{[m-1](l)} \end{cases}$$

$$\text{Weight update rule for the } k\text{-th layer: } \begin{cases} \delta_i^{[k](l)} = \sum_j \delta_j^{[k+1](l)} w_{ji}^{[k+1]} f_i^{[k]'}(x_i^{[k](l)}) \\ \Delta w_{ji}^{[k]} = -\eta \frac{\partial L}{\partial w_{ji}^{[k]}} = \eta \sum_{l=1}^N \delta_j^{[k]} x_i^{(l)} \end{cases}$$

$$\text{Hyperbolic tangent: } \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

Stochastic Gradient Descent: Use a mini-batch at a time instead of the whole batch.

Regularized loss function $L_\lambda(\mathbf{W}; \mathcal{S}) + \frac{\lambda}{2} \sum_{w_i \notin \text{bias}} w_i^2$.

Hence $\Delta w = -\eta \frac{\partial L_\lambda}{\partial w} = -\eta \frac{\partial L}{\partial w} - \eta \lambda w$, where the term $-\eta \lambda w$ is called the weight decay.

Momentum: $\Delta w_{(t)} = -(1 - \beta)\eta \frac{\partial L}{\partial w} + \beta \Delta w_{(t-1)}$, $\beta \in (0.5, 1)$

- Activation Functions

$$\text{ReLU}(x) = \max\{0, x\}$$

$$\text{ReLU}'(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\text{Softplus}(x) = \log(1 + e^x)$$

$$\text{Softplus}'(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\text{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}, \alpha \text{ very small}$$

$$\text{ELU}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

$$\text{ELU}'(x) = \begin{cases} 1 & x \geq 0 \\ \alpha e^x & x < 0 \end{cases}$$

$$\text{SELU}(x) = \lambda \text{ELU}(x)$$

If runtime is not a concern, performance: $\text{ELU} > \text{LeakyReLU} > \text{ReLU} > \tanh > \text{sigmoid}$

- Weight initialization

Xavier initialization: $w_{(0)} \sim \mathcal{N}(0, \sigma^2)$ where $\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ or $w_{(0)} \sim \mathcal{U}([-r, r])$ where $r = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$.

He initialization: $w_{(0)} \sim \mathcal{N}(0, \sigma^2)$ where $\sigma^2 = \frac{4}{n_{\text{in}} + n_{\text{out}}}$ or $w_{(0)} \sim \mathcal{U}([-r, r])$ where $r = \sqrt{\frac{12}{n_{\text{in}} + n_{\text{out}}}}$.

He initialization is for ReLU and its variants, including ELU.

- Batch Normalization

To prevent internal covariate shift, we apply batch normalization: $\text{BN}_{\gamma, \beta} : \mathbf{x}^{(l)} \mapsto \hat{\mathbf{x}}^{(l)}$.

For a mini-batch $\mathcal{B} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$, we perform the following on each feature $i \in \{1, \dots, d\}$.

We standardize $\tilde{x}_i^{(l)} \leftarrow \frac{x_i^{(l)} - \mu_i^{(\mathcal{B})}}{\sqrt{\sigma_i^{2(\mathcal{B})} + \epsilon}}$, where
$$\begin{cases} \mu_i^{(\mathcal{B})} = \frac{1}{m} \sum_{l=1}^m x_i^{(l)} \\ \sigma_i^{2(\mathcal{B})} = \frac{1}{m} \sum_{l=1}^m (x_i^{(l)} - \mu_i^{(\mathcal{B})})^2 \end{cases}$$

The output of the batch-normalization layer: $\hat{x}_i^{(l)} \leftarrow \gamma_i \tilde{x}_i^{(l)} + \beta_i$, where $(\gamma_i, \beta_i)_{i=1}^d$ are learnable parameters.

During prediction, normalization parameters $(\gamma_i, \beta_i)_{i=1}^d$ are based on the whole dataset statistics.

- Dropout

For a dropout layer $\mathbf{z}^{[k-1]} \xrightarrow[p]{\mathbf{W}^{[k],f^{[k]}}} \mathbf{z}^{[k]}, \mathbb{R}^{d_{k-1}} \rightarrow \mathbb{R}^{d_k}$, for each node $z_i^{[k-1]}$, there's a probability p such that $z_i^{[k-1]}$ is set to 0.

$\forall i \in \{1, \dots, d_{k-1}\}, z_i^{[k-1]} = \begin{cases} 0 & \omega_i = 1 \\ \text{unchanged} & \omega_i = 0 \end{cases}, \omega_i \stackrel{\text{i.i.d.}}{\sim} \text{Ber}(p)$, where p is usually set to 0.5.

To address the discrepancy between testing and training, we can perform $\mathbf{w} \leftarrow (1 - p)\mathbf{w}$ after training, or $\mathbf{z}^{[k]} \leftarrow (1 - p)\mathbf{z}^{[k]}$ during training

Data augmentation: Translation, rotation, scaling; Lighting; Cropping; Flipping

Element-wise Operations:

Hadamard product: $\begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} \odot \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ \vdots \\ a_n b_n \end{bmatrix}$. Hadamard division: $\begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} \oslash \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} a_1/b_1 \\ \vdots \\ a_n/b_n \end{bmatrix}$

- Optimization Methods

Stochastic gradient descent (SGD): $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} L$

AdaGrad uses an adaptive learning rate which decays faster for steeper dimensions.

For each w_i , $\begin{cases} s_i \leftarrow s_i + (\frac{\partial L}{\partial w_i})^2 \\ w_i \leftarrow w_i - \frac{\eta}{\sqrt{s_i + \epsilon_i}} \cdot \frac{\partial L}{\partial w_i} \end{cases}$. Or in vector form, $\begin{cases} \mathbf{s} \leftarrow \mathbf{s} + (\nabla_{\mathbf{w}} L)^2 \\ \mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{\nabla_{\mathbf{w}} L}{\sqrt{\mathbf{s} + \epsilon}} \end{cases}$

where $(\cdot)^2, \div, \sqrt{\cdot}$ refer to elementwise operations.

RMSProp introduces exponential decay to prevent AdaGrad from decaying too fast.

For each w_i , $\begin{cases} s_i \leftarrow \beta s_i + (1 - \beta)(\frac{\partial L}{\partial w_i})^2 \\ w_i \leftarrow w_i - \frac{\eta}{\sqrt{s_i + \epsilon_i}} \cdot \frac{\partial L}{\partial w_i} \end{cases}$. Or in vector form, $\begin{cases} \mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta)(\nabla_{\mathbf{w}} L)^2 \\ \mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \frac{\nabla_{\mathbf{w}} L}{\sqrt{\mathbf{s} + \epsilon}} \end{cases}$

Exponential decay is in the sense that $\mathbf{s}_{(t)} = (1 - \beta)(\mathbf{g}_{(t)}^2 + \beta \mathbf{g}_{(t-1)}^2 + \dots + \beta^{t-2} \mathbf{g}_{(2)}^2 + \beta^{t-1} \mathbf{g}_{(1)}^2) + \beta^t \mathbf{s}_{(0)}$, where $\mathbf{g}_{(t)} = \nabla_{\mathbf{w}} L$ in the t -th iteration.

Adam (Adaptive moment estimation) combines the idea of momentum and RMSProp.

For each w_i , $\begin{cases} s_i \leftarrow \frac{\beta_2 s_i + (1 - \beta_2)(\frac{\partial L}{\partial w_i})^2}{1 - \beta_2^t} \\ \Delta w_i \leftarrow \frac{\beta_1 \Delta w_i + (1 - \beta_1) \frac{\partial L}{\partial w_i}}{1 - \beta_1^t} \\ w_i \leftarrow w_i + \eta \cdot \frac{\Delta w_i}{\sqrt{s_i + \epsilon_i}} \end{cases}$. Or in vector form, $\begin{cases} \mathbf{s} \leftarrow \frac{\beta_2 \mathbf{s} + (1 - \beta_2)(\nabla_{\mathbf{w}} L)^2}{1 - \beta_2^t} \\ \Delta \mathbf{w} \leftarrow \frac{\beta_1 \Delta \mathbf{w} + (1 - \beta_1) \nabla_{\mathbf{w}} L}{1 - \beta_1^t} \\ \mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \frac{\Delta \mathbf{w}}{\sqrt{\mathbf{s} + \epsilon}} \end{cases}$

- Convolutional Neural Networks (CNN)

$$\mathbf{z}^{[k]} = f^{[k]}(\mathbf{z}^{[k-1]} * \mathbf{K}^{[k]})$$

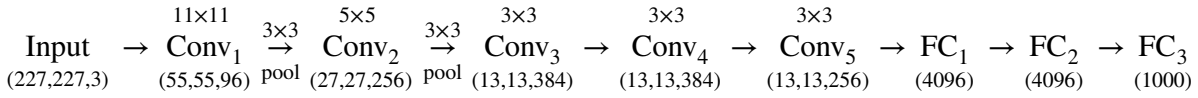
Kernel size: $f \times f$, Length of one side of the input feature map: n , Padding size: p , Stride: s

$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} - 2p - f}{s} + 1 \right\rfloor$$

A pooling layer subsamples the input image to give a smaller image.

Typical CNN Architectures:

AlexNet (2012)



VGGNet (2014): Simpler but deeper architecture

GoogLeNet (2014): Applies inception module

$$\text{Inception module: Previous Layer} \rightarrow \left\{ \begin{array}{l} 1 \times 1 \text{ Conv} \\ 1 \times 1 \text{ Conv} \rightarrow 3 \times 3 \text{ Conv} \\ 1 \times 1 \text{ Conv} \rightarrow 5 \times 5 \text{ Conv} \\ 3 \times 3 \text{ MaxPool} \rightarrow 1 \times 1 \text{ Conv} \end{array} \right\} \text{Concat} \rightarrow \text{Next Layer}$$

Adding 1×1 Conv layers can significantly reduce the number of parameters.

ResNet (2015): Applies skip connections

$$\mathbf{x}^{[k]} = \mathcal{F}^{[k]}(\mathbf{x}^{[k-1]}) + \mathbf{x}^{[k-1]}$$

DenseNet: Inspired by ResNet's success, let each layer takes all preceding feature maps as input.

$$\mathbf{x}^{[k]} = \mathcal{F}^{[k]}(\text{concat}(\mathbf{x}^{[0]}, \mathbf{x}^{[1]}, \dots, \mathbf{x}^{[k-1]}))$$

Transition layers:

Input $\rightarrow$ Conv $\rightarrow$ DenseBlock₁ $\rightarrow$ Conv & Pool $\rightarrow$ DenseBlock₂ $\rightarrow$ Conv & Pool $\rightarrow$ DenseBlock₃ $\rightarrow$ Pool

Layers in each DenseBlock is a DenseNet and have feature maps of the same size.

Layers between adjacent DenseBlock are called transition layers to change the feature map sizes.

- Recurrent Neural Networks (RNN)

For sequence processing, it's not good to use traditional MLPs to process variable-length input sequences. Hence we introduce RNN.

$$\begin{cases} \mathbf{s}_t = f(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1} + \mathbf{b}) \\ \mathbf{y}_t = g(\mathbf{V}\mathbf{s}_t) \end{cases}, \text{ or equivalently, } \begin{cases} \mathbf{s} \leftarrow f(\mathbf{U}\mathbf{x} + \mathbf{W}\mathbf{s} + \mathbf{b}) \\ \mathbf{y} \leftarrow g(\mathbf{V}\mathbf{s}) \end{cases}$$

The network parameters $\mathbf{U}, \mathbf{V}, \mathbf{W}, \mathbf{b}$ are shared across time steps.

Sequence-to-sequence network (Conventional RNN architecture): Input and output sequences have the same length and align in each time step.

Sequence-to-vector network (encoder): $\mathbf{y}_1, \dots, \mathbf{y}_{n-1}$ are ignored. The sequence $(\mathbf{x}_i)_{i=1}^n$ is encoded by $\mathbf{y}_n$.

Vector-to-sequence network (decoder): There is only a single input $\mathbf{x}_1$ at the first step. The output becomes the input of the next time step, i.e., $\mathbf{x}_t = \mathbf{y}_{t-1}$. Through these iterations we get a sequence $(\mathbf{y}_i)_{i=1}^n$.

Encoder-Decoder RNN architecture: composed of an encoder followed by a decoder, and the input sequence and output sequence doesn't necessarily have the same length.

Training method: Backpropagation through time (BPTT) by unrolling it through time. The loss function is defined using all the outputs. Since the weights are shared, only the average version will be used for weight update.

When training an RNN, exploding gradient problem may occur.

We can apply gradient clipping $\tilde{\mathbf{g}} = \begin{cases} \mathbf{g} & \|\mathbf{g}\| \leq \theta \\ \frac{\theta}{\|\mathbf{g}\|} \mathbf{g} & \|\mathbf{g}\| > \theta \end{cases}$ which controls the magnitude of the gradient when it exceeds a certain threshold θ .

We can also use Truncated BPTT, which truncates a long sequence into multiple shorter subsequence.

- Long Short-Term Memory (LSTM)

Faster convergence and more capable of detecting long-term dependencies in the sequences.

Each cell has 2 states: long-term state $\mathbf{c}_t$ and short-term state $\mathbf{h}_t$. The interplay between $\mathbf{c}_t$ and $\mathbf{h}_t$ is achieved through 3 gates in an LSTM cell: Forget gate, Input gate and Output gate.

$$\begin{cases} \mathbf{f}_t = \sigma(\mathbf{W}_{fx}\mathbf{x}_t + \mathbf{W}_{fh}\mathbf{h}_{t-1} + \mathbf{b}_f) & (\text{forget mask}) \\ \mathbf{i}_t = \sigma(\mathbf{W}_{ix}\mathbf{x}_t + \mathbf{W}_{ih}\mathbf{h}_{t-1} + \mathbf{b}_i) & (\text{input mask}) \\ \mathbf{o}_t = \sigma(\mathbf{W}_{ox}\mathbf{x}_t + \mathbf{W}_{oh}\mathbf{h}_{t-1} + \mathbf{b}_o) & (\text{output mask}) \\ \tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_{cx}\mathbf{x}_t + \mathbf{W}_{ch}\mathbf{h}_{t-1} + \mathbf{b}_{\tilde{c}}) & (\text{info to be added to } \mathbf{c}_t) \end{cases} \implies \begin{cases} \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\ \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \end{cases}$$

- Gated Recurrent Unit (GRU)

The gated recurrent unit (GRU) simplifies LSTM by merging two state vectors into $\mathbf{h}_t$, and applying only one gate controller $\mathbf{z}_t$ controlling both the forget gate and the input gate.

$$\begin{cases} \mathbf{z}_t = \sigma(\mathbf{W}_{zx}\mathbf{x}_t + \mathbf{W}_{zh}\mathbf{h}_{t-1} + \mathbf{b}_z) & (\text{update mask}) \\ \mathbf{r}_t = \sigma(\mathbf{W}_{rx}\mathbf{x}_t + \mathbf{W}_{rh}\mathbf{h}_{t-1} + \mathbf{b}_r) & (\text{reset mask}) \\ \tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_{\tilde{h}}) & (\text{info to be added to } \mathbf{h}_t) \end{cases} \implies \mathbf{h}_t = (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

- Attention

The ordinary RNN encoder-decoder architecture is forgetful. Hence we introduce the attention mechanism, which is located between the encoder and the decoder.

We have the input sequence $(\mathbf{x}_i)_{i=1}^n$, the corresponding state of the encoder $(\mathbf{h}_i)_{i=1}^n$, and the initial state of the

decoder $\mathbf{s}_0$. For each time step $t = 1, 2, \dots$, we perform
$$\left\{ \begin{array}{l} \mathbf{c}_t = \sum_{i=1}^n \underbrace{\text{softmax}_{i=1, \dots, n} \left(\overbrace{\text{FC} \left(\left[\begin{array}{c} \mathbf{s}_{t-1} \\ \mathbf{h}_i \end{array} \right] \right)}^{e_{t,i}} \right)}^{\alpha_{t,i}} \mathbf{h}_i, \text{ and we} \\ \text{Decoder}_{\mathbf{s}_{t-1}}(\mathbf{c}_t) = \mathbf{y}_t, \text{ with new state } \mathbf{s}_t \end{array} \right.$$

get the output sequence $(\mathbf{y}_j)_{j=1}^m$.

The RNN prediction error terms are propagated backwards through the decoder, then $\mathbf{W}$, and finally the encoder.

- Principal Component Analysis (PCA)

Dimensionality can be very large. We want to reduce it.

Feature selection: From d features, choose $k < d$ features and discard the rest.

Feature transformation: Find a mapping $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$. Can be supervised or unsupervised.

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction method which find a linear projection $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$ with minimum loss of data variance.

Assume for a data set $\mathcal{S} = \{\mathbf{x}^{(l)}\}_{l=1}^N$, we project every $\mathbf{x}^{(l)} \in \mathbb{R}^d$ onto a unit vector $\mathbf{w} \in \mathbb{R}^d$. The component of $\mathbf{x}^{(l)}$ in the direction of $\mathbf{w}$ should be $z^{(l)} = \mathbf{w}^\top \mathbf{x}^{(l)}$. Hence $\text{Var}(z) = \text{Var}(\mathbf{w}^\top \mathbf{x})$ is what we want to maximize.

Let $\mu = \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_d \end{bmatrix} = E[\mathbf{x}]$. We can get the covariance matrix $\mathbf{C} = E[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^\top] \in \mathbb{R}^{d \times d}$.

$$\mathbf{C} = \frac{1}{n-1} (\mathbf{X} - \bar{\mathbf{X}})^\top (\mathbf{X} - \bar{\mathbf{X}}), \text{ where } \bar{\mathbf{X}} = \mathbf{1}_N \bar{\mathbf{x}}^\top, \text{ where } \bar{\mathbf{x}}^\top = \begin{bmatrix} \frac{1}{N} \sum_{i=1}^N x_{i1} & \dots & \frac{1}{N} \sum_{i=1}^N x_{id} \end{bmatrix}$$

By mathematics, we have $\text{Var}(z) = \mathbf{w}^\top \mathbf{C} \mathbf{w}$.

We want to solve $\max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{w}^\top \mathbf{C} \mathbf{w})$. By Lagrange's method, we have

$$\max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{w}^\top \mathbf{C} \mathbf{w}) \iff \exists \alpha \in \mathbb{R}, \mathbf{C} \mathbf{w} = \alpha \mathbf{w} \iff \alpha \text{ is the largest eigenvalue of } \mathbf{C} \text{ with eigenvector } \mathbf{w}$$

Let $\mathbf{W} = [\mathbf{w}_1 \dots \mathbf{w}_k] \in \mathbb{R}^{d \times k}$ be the projection matrix formed by k leading eigenvectors. $\mathbf{C}$ being SPSPD guarentees $\mathbf{W}$ being orthogonal. We have $\mathbf{z}^{(l)} = \mathbf{W}^\top (\mathbf{x}^{(l)} - \bar{\mathbf{x}})$.

To avoid variances of different original dimensions vary considerably, we can let preprocess the data so that each dimension has zero mean and unit variance, or choosing leading eigenvectors from the correlation matrix, which are equivalent.

To determine k , we define proportion of variance a, where $\lambda_1 \geq \dots \geq \lambda_d \geq 0$ are the eigenvalues of $\mathbf{C}$. We can set a threshold θ and set $k = \arg_k \min_k \text{Pov}(k) \geq \theta$.

If the dimensions of the original data are highly correlated, $\mathbf{C}$ would only have a small number of large eigenvalues, and we would have $k \ll d$.

- Autoencoder

Instead of using PCA, we can use autoencoders for dimensionality reduction, which is a special feedforward neural network with a bottle neck layer: $\mathbf{x} \xrightarrow[\text{Encoder}]{f} \mathbf{z} \xrightarrow[\text{Decoder}]{g} \mathbf{x}'$, with $\mathbf{x} \in \mathbb{R}^d, \mathbf{z} \in \mathbb{R}^h, h < d$.

An autoencoder is trained in a self-supervised manner using unlabeled data. A loss function L is needed to quantify the discrepancy between $\mathbf{x}$ and $\mathbf{x}'$.

If f and g are linear and the loss function $L(\mathbf{x}) = \|g(f(\mathbf{x})) - \mathbf{x}\|^2$, then the code $\mathbf{z}$ learned spans the same h -dimensional subspace as PCA.

Autoencoders need no eigendecomposition and hence is more efficient for large d .

Deep autoencoders are autoencoders with more layers added to the encoder and decoder.

We can use autoencoders for denoising tasks.

In a convolutional autoencoder, the encoder consists of convolutional and pooling layers, while the decoder consists of deconvolutional and unpooling layers.

Deconvolution: Every value in the $n \times n$ feature map is duplicated to give a $k \times k$ patch, and the ordinary convolution (with padding) is applied to the dialated $kn \times kn$ map.

We can generalizing convolutional autoencoders for image segmentation, if the target output is not the input itself but a segmentation mask. The loss L is defined as binary classification for each pixel.

U-net architecture

- Clustering

Feature-based clustering: Input is an $N \times d$ feature matrix.

Similarity-based clustering: Input is an $N \times N$ distance matrix.

Partitional clustering: Output is a partition consisting of disjoint sets of examples.

Hierachical clustering: Output is a nested tree of partitions (called dendrogram), and doesn't need the number of clusters to be specified.

- k -means clustering

Task: Given a set $\mathcal{S} = \{\mathbf{x}^{(l)}\}_{l=1}^N$ of unlabeled examples. We want k clusters.

We define the sum of squared errors (SSE)

$$E(\{\mathbf{m}_i\}_{i=1}^k, \mathcal{S}) = \sum_{l=1}^N \sum_{i=1}^k b_i^{(l)} \|\mathbf{x}^{(l)} - \mathbf{m}_i\|^2, \text{ where } b_i^{(l)} = \begin{cases} 1 & i = \arg_j \min_j \|\mathbf{x}^{(l)} - \mathbf{m}_j\| \\ 0 & \text{otherwise} \end{cases}$$

k -means algorithm goes as follows:

```

Initialize  $\mathbf{m}_1, \dots, \mathbf{m}_k$ 
Repeat
  For each  $\mathbf{x}^{(l)} \in \mathcal{S}$ :
     $b_i^{(l)} \leftarrow \begin{cases} 1 & i = \arg_j \min_j \|\mathbf{x}^{(l)} - \mathbf{m}_j\| \\ 0 & \text{otherwise} \end{cases}$ 
  For  $\mathbf{m}_1, \dots, \mathbf{m}_k$ :
     $\mathbf{m}_i \leftarrow \frac{\sum_{l=1}^N b_i^{(l)} \mathbf{x}^{(l)}}{\sum_{l=1}^N b_i^{(l)}}$ 
Until  $\mathbf{m}_1, \dots, \mathbf{m}_k$  stop changing

```

Only local convergence is guaranteed.

To determine k , we use the elbow method. Plot the SSE- k plot.

Variations of k -means

k -medians clustering: Median instead of mean is used to determine the centroid for each center.

Advantage: Less sensitive to outliers

k -medoid clustering

- Hierarchical Clustering

Two approaches: agglomerative (or bottom-up) approach and divisive (or top-down) approach

The proximity between clusters can be defined using a function $D(C_i, C_j)$ between two clusters C_i, C_j .

Single-link proximity $D_s(C_i, C_j) = \min_{\substack{\mathbf{x}^{(l_1)} \in C_i \\ \mathbf{x}^{(l_2)} \in C_j}} d(\mathbf{x}^{(l_1)}, \mathbf{x}^{(l_2)})$ measures the shortest distance of two clusters.

Complete-link proximity $D_c(C_i, C_j) = \max_{\substack{\mathbf{x}^{(l_1)} \in C_i \\ \mathbf{x}^{(l_2)} \in C_j}} d(\mathbf{x}^{(l_1)}, \mathbf{x}^{(l_2)})$ measures the longest distance of two clusters.

Average-link proximity $D_a(C_i, C_j) = \frac{1}{|C_i| \cdot |C_j|} \sum_{\substack{\mathbf{x}^{(l_1)} \in C_i \\ \mathbf{x}^{(l_2)} \in C_j}} d(\mathbf{x}^{(l_1)}, \mathbf{x}^{(l_2)})$

Agglomerative hierarchical clustering algorithm

Given the distance matrix $\mathbf{D} \in \mathbb{R}^{N \times N}$

Repeat

 Merge the closest pair of clusters

 Update the distance (proximity) matrix (The size of the matrix should be one less.)

Until only one cluster remains

Having multiple closest pairs of clusters may lead to the same or different tree structures.

- Generative Adversarial Network (GAN)

In generative modeling, we train a network to model a data distribution.

Data generation and density estimation are inverse operations of each other.

Implicit generative model

A code vector $\mathbf{z}$ is sampled from an explicit, fixed, simple distribution $p(\mathbf{z})$, e.g. $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$.

A generator is a differentiable function $G : \mathbf{z} \mapsto \hat{\mathbf{x}}$, code space $\rightarrow$ data space.

To train the implicit generative model, we need loss function, i.e., a criterion for evaluating the quality of generated images.

A generative adversarial network (GAN) consists of a generator and a discriminator that compete with each other.

The discriminator $D : \mathbf{x} \mapsto r \in (0,1)$ outputs the predicted probability that $\mathbf{x}$ is real.

Cost function for the discriminator $J_D = -E_{\mathbf{x} \sim \mathcal{D}} [\log D(\mathbf{x})] - E_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$, where $\mathcal{D}$ is the set of real images. The expectation terms are approximated by the empirical means.

Cost function for the generator $J_G = -J_D = \text{const} + E_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$

$J_G + J_D = 0$ is a zero sum game and can be formulated as a minimax optimization problem $\max_G \min_D J_D$.

The cost function J_G suffers saturation when $D(\mathbf{x})$ is small. We can use $\tilde{J}_G = -E_{\mathbf{z} \sim p(\mathbf{z})} [\log D(G(\mathbf{z}))]$. In this case, $\tilde{J}_G + J_D \neq 0$ and it becomes heuristic and is no longer zero-sum.

Conditional GAN: $\max_G \min_D J_D = -E_{\mathbf{x} \sim \mathcal{D}} [\log D(\mathbf{x} | \mathbf{y})] - E_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z} | \mathbf{y})))]$

If $\mathbf{y}$ is an image, the conditional GAN can be used for image-to-image translation, e.g., style transfer, image colorization, image super-resolution.

To perform image-to-image translation tasks, we can use CycleGAN, which consists of two generators going from style 1 to style 2 and vice versa, i.e., $\mathbf{x} \xrightarrow{G} \mathbf{y}$, and two corresponding discriminator D_X, D_Y

Each generator should be cycle-consistent, i.e., should minimize the reconstruction losses $\|F(G(\mathbf{x})) - \mathbf{x}\|$ and $\|G(F(\mathbf{y})) - \mathbf{y}\|$. Also we should minimize the discriminator loss.

- Transformers

The input of the encoder is $\mathbf{X} \in \mathbb{R}^{n \times d}$, where every row is a word vector with dimension d .

We first add the position encoding $\mathbf{X}^{[0]} = \mathbf{X} + \mathbf{P}_{n \times d}$.

For each encoder $t = 1, 2, \dots, N$:

For each attention head $i = 1, 2, \dots, h$:

$$\mathbf{Z}_i^{[t]} = \text{softmax}_{\text{by row}} \left(\frac{\mathbf{Q}_{X,i}^{[t]} \mathbf{K}_{X,i}^{[t]\top}}{\sqrt{d_K}} \right) \mathbf{V}_{X,i}^{[t]} \in \mathbb{R}^{n \times d_V}, \text{ where } \begin{cases} \mathbf{Q}_{X,i}^{[t]} = \mathbf{X}^{[t-1]} \mathbf{W}_i^{Q_X[t]} \in \mathbb{R}^{n \times d_K} \\ \mathbf{K}_{X,i}^{[t]} = \mathbf{X}^{[t-1]} \mathbf{W}_i^{K_X[t]} \in \mathbb{R}^{n \times d_K} \\ \mathbf{V}_{X,i}^{[t]} = \mathbf{X}^{[t-1]} \mathbf{W}_i^{V_X[t]} \in \mathbb{R}^{n \times d_V} \end{cases}$$

$$(\text{Remarks: } \begin{cases} \mathbf{W}_i^{Q_X[t]} \in \mathbb{R}^{d \times d_K} \\ \mathbf{W}_i^{K_X[t]} \in \mathbb{R}^{d \times d_K} \\ \mathbf{W}_i^{V_X[t]} \in \mathbb{R}^{d \times d_V} \end{cases}, \begin{cases} \mathbf{W}_i^{Q_Y[t]} \in \mathbb{R}^{d \times d_K} \\ \mathbf{W}_i^{K_Y[t]} \in \mathbb{R}^{d \times d_K} \\ \mathbf{W}_i^{V_Y[t]} \in \mathbb{R}^{d \times d_V} \end{cases})$$

$$\mathbf{Z}^{[t]} = \begin{bmatrix} \mathbf{Z}_1^{[t]} & \mathbf{Z}_2^{[t]} & \dots & \mathbf{Z}_h^{[t]} \end{bmatrix} \mathbf{W}^{O_X[t]} \in \mathbb{R}^{n \times d}, \text{ where } \mathbf{W}^{O_X[t]} \in \mathbb{R}^{hd_V \times d}$$

$$\tilde{\mathbf{Z}}^{[t]} = \text{LayerNorm}(\mathbf{Z}^{[t]} + \mathbf{X}^{[t-1]})$$

$$\mathbf{Z}'^{[t]} = \left(\text{ReLU}(\tilde{\mathbf{Z}}^{[t]} \mathbf{W}_{X,1}^{[t]} + \mathbf{b}_{X,1}^{[t]}) \right) \mathbf{W}_{X,2}^{[t]} + \mathbf{b}_{X,2}^{[t]}, \text{ where } \mathbf{W}_{X,1}^{[t]} \in \mathbb{R}^{d \times d_I}, \mathbf{W}_{X,2}^{[t]} \in \mathbb{R}^{d_I \times d}.$$

$$\mathbf{X}^{[t]} = \text{LayerNorm}(\mathbf{Z}'^{[t]} + \tilde{\mathbf{Z}}^{[t]})$$

And we get $\mathbf{X}^{[N]}$, which is the output of the encoders.

The input of the decoder is $\mathbf{Y} \in \mathbb{R}^{m \times d}$, where every row is a word vector with dimension d .

We first add the position encoding $\mathbf{Y}^{[0]} = \mathbf{Y} + \mathbf{P}_{m \times d}$.

For each decoder $t = 1, 2, \dots, N$:

For each attention head $i = 1, 2, \dots, h$:

$$\mathbf{T}_i^{[t]} = \underset{\text{by row}}{\text{softmax}} \left(\frac{\mathbf{Q}_{Y,i}^{[t]} \mathbf{K}_{Y,i}^{[t]\top}}{\sqrt{d_K}} + \mathbf{M} \right) \mathbf{V}_{Y,i}^{[t]} \in \mathbb{R}^{m \times d_V}, \text{ where } \begin{cases} \mathbf{Q}_{Y,i}^{[t]} = \mathbf{Y}^{[t-1]} \mathbf{W}_i^{Q_Y[t]} \in \mathbb{R}^{m \times d_K} \\ \mathbf{K}_{Y,i}^{[t]} = \mathbf{Y}^{[t-1]} \mathbf{W}_i^{K_Y[t]} \in \mathbb{R}^{m \times d_K} \\ \mathbf{V}_{Y,i}^{[t]} = \mathbf{Y}^{[t-1]} \mathbf{W}_i^{V_Y[t]} \in \mathbb{R}^{m \times d_V} \end{cases}$$

$$(\mathbf{M} \in \mathbb{R}^{m \times m} \text{ with } m_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases} \text{ is a mask})$$

$$\mathbf{T}^{[t]} = \begin{bmatrix} \mathbf{T}_1^{[t]} & \mathbf{T}_2^{[t]} & \dots & \mathbf{T}_h^{[t]} \end{bmatrix} \mathbf{W}^{O_Y[t]} \in \mathbb{R}^{m \times d}, \text{ where } \mathbf{W}^{O_Y[t]} \in \mathbb{R}^{hd_V \times d}$$

$$\tilde{\mathbf{T}}^{[t]} = \text{LayerNorm}(\mathbf{T}^{[t]} + \mathbf{Y}^{[t-1]})$$

For each attention head $i = 1, 2, \dots, h$:

$$\hat{\mathbf{T}}_i^{[t]} = \underset{\text{by row}}{\text{softmax}} \left(\frac{\mathbf{Q}_{Y,i}^{[t]} \mathbf{K}_{X,i}^{[N]\top}}{\sqrt{d_K}} \right) \mathbf{V}_{X,i}^{[N]} \in \mathbb{R}^{m \times d_V}, \text{ where } \begin{cases} \mathbf{Q}_{Y,i}^{[t]} = \tilde{\mathbf{T}}^{[t]} \mathbf{W}_i^{Q_Y[t]} \in \mathbb{R}^{m \times d_K} \\ \mathbf{K}_{X,i}^{[N]} = \mathbf{X}^{[N]} \mathbf{W}_i^{K_X[N]} \in \mathbb{R}^{n \times d_K} \\ \mathbf{V}_{X,i}^{[N]} = \mathbf{X}^{[N]} \mathbf{W}_i^{V_X[N]} \in \mathbb{R}^{n \times d_V} \end{cases}$$

$$\hat{\mathbf{T}}^{[t]} = \begin{bmatrix} \hat{\mathbf{T}}_1^{[t]} & \hat{\mathbf{T}}_2^{[t]} & \dots & \hat{\mathbf{T}}_h^{[t]} \end{bmatrix} \mathbf{W}^{O_{XY}[t]} \in \mathbb{R}^{m \times d}, \text{ where } \mathbf{W}^{O_{XY}[t]} \in \mathbb{R}^{hd_V \times d}$$

$$\tilde{\tilde{\mathbf{T}}}^{[t]} = \text{LayerNorm}(\hat{\mathbf{T}}^{[t]} + \tilde{\mathbf{T}}^{[t]})$$

$$\mathbf{T}'^{[t]} = \left(\text{ReLU}(\tilde{\tilde{\mathbf{T}}}^{[t]} \mathbf{W}_{Y,1}^{[t]} + \mathbf{b}_{Y,1}^{[t]}) \right) \mathbf{W}_{Y,2}^{[t]} + \mathbf{b}_{Y,2}^{[t]}, \text{ where } \mathbf{W}_{Y,1}^{[t]} \in \mathbb{R}^{d \times d_I}, \mathbf{W}_{Y,2}^{[t]} \in \mathbb{R}^{d_I \times d}$$

$$\mathbf{Y}^{[t]} = \text{LayerNorm}(\mathbf{T}'^{[t]} + \tilde{\tilde{\mathbf{T}}}^{[t]})$$

And we get $\mathbf{Y}^{[N]}$. We have $\hat{\mathbf{Y}} = \text{argmax}(\text{softmax}(\text{Linear}(\mathbf{Y}^{[N]})))$

BERT (Bidirectional Encoder Representations from Transformers)

- Support Vector Machine (SVM)

Given a dataset $\{\mathbf{x}^{(l)}\}_{l=1}^N$ that is linearly separable. We want to find an optimal separating hyperplane $H : \mathbf{w}^\top \mathbf{x} + w_0 = 0$.

We scale $\mathbf{w}$ and w_0 properly such that the points closest to H satisfy $|\mathbf{w}^\top \mathbf{x} + w_0| = 1$. Such a hyperplane is called a canonical separating hyperplane.

Margin of H refers to the distance of the closest data point to H . We want to maximize the margin. By easy computation, we can show that the margin $\gamma = \frac{1}{\|\mathbf{w}\|}$. Hence we want to maximize $\|\mathbf{w}\|$.

Given a labeled dataset $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^N$ where $y^{(l)} = \pm 1$, we want $\mathbf{w}, w_0$ to satisfy

$$\mathbf{w}^\top \mathbf{x}^{(l)} + w_0 \begin{cases} \geq 1 & y^{(l)} = 1 \\ \leq -1 & y^{(l)} = -1 \end{cases}, \text{ or equivalently, } y^{(l)}(\mathbf{w}^\top \mathbf{x}^{(l)} + w_0) \geq 1.$$

Primal optimization problem: $\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2$ subject to $y^{(l)}(\mathbf{w}^\top \mathbf{x}^{(l)} + w_0) \geq 1 \ \forall l$

It's more convenient to solve the dual problem (time: $O(N^3)$, space: $O(N^2)$) rather than the primal problem directly ($O(\sim d)$) when $N \ll d$.

By introducing N Lagrange multipliers $\{\alpha_l\}_{l=1}^N$, we can turn the primal problem into a dual problem.

$$\text{Dual optimization problem: } \max_{\alpha_1, \dots, \alpha_N} \sum_{l=1}^N \alpha_l - \frac{1}{2} \sum_{l=1}^N \sum_{l'=1}^N \alpha_l \alpha_{l'} y^{(l)} y^{(l')} \mathbf{x}^{(l)\top} \mathbf{x}^{(l')} \text{ subject to } \begin{cases} \sum_{l=1}^N \alpha_l y^{(l)} = 0 \\ \alpha_l \geq 0 \end{cases} \forall l$$

At the optimal solution, most $\alpha_l = 0$. The vectors $\mathbf{x}^{(l)}$ with $\alpha_l > 0$ are called support vectors, usually a small subset $\mathcal{V} = \{\mathbf{x}^{(l)} : \alpha_l > 0\} \subset \mathcal{S}$ of all training data points. For the points lying beyond the margin, their exact location are irrelevant.

From the derivation of the dual problem, we get $\hat{\mathbf{w}} = \sum_{\mathbf{x}^{(l)} \in \mathcal{V}} \alpha_l y^{(l)} \mathbf{x}^{(l)}$, $\hat{w}_0 = \frac{1}{|\mathcal{V}|} \sum_{\mathbf{x}^{(l)} \in \mathcal{V}} y^{(l)} - \hat{\mathbf{w}}^\top \mathbf{x}^{(l)}$. All $\mathbf{x}^{(l)} \in \mathcal{V}$ are used to compute $\hat{w}_0$ for numerical stability.

Hence the discriminant function $g(\mathbf{x}) = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{w}_0$.

$$\text{For test data } \mathbf{x}^*, \text{ we predict } y^* = \begin{cases} 1 & g(\mathbf{x}^*) > 0 \\ -1 & g(\mathbf{x}^*) \leq 0 \end{cases}$$

For K -class problems, we can define K two-class problems with $g_i(\mathbf{x}) = \hat{\mathbf{w}}_i^\top \mathbf{x} + \hat{w}_{0,i}$, $i = 1, \dots, K$.

For a test data $\mathbf{x}^*$, we predict $y^* = \arg \max_{1 \leq i \leq K} g_i(\mathbf{x}^*)$.

However, due to outliers and noises, the optimized hyperplane may not be the best solution, or even not exist.

A soft-margin SVM allows violating the constraint $y^{(l)}(\mathbf{w}^\top \mathbf{x}^{(l)} + w_0) \geq 1$ by introducing slack variables $\xi_l \geq 0$, $l = 1, \dots, N$. The relaxed separation constraints: $y^{(l)}(\mathbf{w}^\top \mathbf{x}^{(l)} + w_0) \geq 1 - \xi_l$.

We should penalize ξ_l , with $\xi_l = 0$ no penalty, $0 < \xi_l < 1$ small penalty, and $\xi_l \geq 1$ large penalty.

$$\text{Primal optimization problem: } \min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{l=1}^N \xi_l \text{ subject to } \begin{cases} y^{(l)}(\mathbf{w}^\top \mathbf{x}^{(l)} + w_0) \geq 1 - \xi_l \\ \xi_l \geq 0 \end{cases} \forall l.$$

$$\text{Dual optimization problem: } \max_{\alpha_1, \dots, \alpha_N} \sum_{l=1}^N \alpha_l - \frac{1}{2} \sum_{l=1}^N \sum_{l'=1}^N \alpha_l \alpha_{l'} y^{(l)} y^{(l')} \mathbf{x}^{(l)\top} \mathbf{x}^{(l')} \text{ subject to } \begin{cases} \sum_{l=1}^N \alpha_l y^{(l)} = 0 \\ 0 \leq \alpha_l \leq C \end{cases} \forall l.$$

We can also view the hard-/soft-margin SVMs as unconstrained optimization problems. Each optimization problem minimizes a loss function with a regularizer.

$$\text{Objective function for hard-margin SVM: } J(\mathbf{x}; \mathbf{w}) = \sum_{l=1}^N E_\infty(y^{(l)} g(\mathbf{x}^{(l)})) + \lambda \|\mathbf{w}\|^2, \text{ where}$$

$$E_\infty(z) = \begin{cases} 0 & z \geq 1 \\ \infty & z < 1 \end{cases}$$

$$\text{Objective function for soft-margin SVM: } J(\mathbf{x}; \mathbf{w}) = \sum_{l=1}^N E_h(y^{(l)} g(\mathbf{x}^{(l)})) + \lambda \|\mathbf{w}\|^2, \text{ where}$$

$$E_h(z) = \max\{0, 1 - z\} \text{ is called the hinge loss.}$$

Kernel methods: Instead of defining model in the original input space, the problem is mapped to a new space by performing a nonlinear transformation. A linear model is then applied in the new space.

Basis functions $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$, $\mathbf{z} = \Phi(\mathbf{x})$, where $z_i = \phi_i(\mathbf{x})$, $i = 1, \dots, k$

Discriminant function $g(\mathbf{x}) = \mathbf{w}^\top \Phi(\mathbf{x}) = \sum_{i=1}^k w_i \phi_i(\mathbf{x})$. We set $z_1 := \phi_1(\mathbf{x}) \equiv 1$ as the role of w_0 .

Dual optimization problem: $\max_{\alpha_1, \dots, \alpha_N} \sum_{l=1}^N \alpha_l - \frac{1}{2} \sum_{l=1}^N \sum_{l'=1}^N \alpha_l \alpha_{l'} y^{(l)} y^{(l')} \underbrace{\Phi(\mathbf{x}^{(l)})^\top \Phi(\mathbf{x}^{(l')})}_{K(\mathbf{x}^{(l)}, \mathbf{x}^{(l')})}$

For a training set $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^N$, we define an $N \times N$ kernel matrix. All kernel matrices are SPSD.

We define the kernel function $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ directly.

Polynomial kernel $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + 1)^q$, where q is the degree.

Radial basis function kernel (Gaussian kernel): $K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2s^2}\right)$, which can be seen as a

similarity measure. It can be generalized to $K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{d(\mathbf{x}, \mathbf{x}')}{2s^2}\right)$, where d is some distance function.

- Decision Tree

Decision tree can be used for classification and regression (not discussed here).

Two types of node:

Decision node: a test function with branches

Leaf node: value contributes the output.

Decision tree learning is a recursive tree construction procedure with the divide-and-conquer approach.

Problem formulation: Dataset $\mathcal{S}_0 = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^N$, where each $\mathbf{x}^{(l)} = \begin{bmatrix} x_1^{(l)} \\ \vdots \\ x_r^{(l)} \end{bmatrix} \in \mathbb{R}^r$ has r attributes

$A = \{1, \dots, r\}$. For each attribute $a \in A$, $x_a^{(l)} \in \{1, \dots, K_a\}$ has K_a classes, and $y^{(l)} \in \{1, \dots, K\}$ has K target classes.

For each node $(\mathcal{S}, A', k_0)$, we have examples $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^n$, the set of attributes not yet used $A' \subseteq A$, and a default class label $k_0 \in \{1, \dots, K\}$.

In every step (every time down-branching a node), we should choose the attribute $a \in A'$ such that it reduces the impurity to the fullest.

We can use entropy as a measure of impurity.

Let p_k denotes the proportion of $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^n$ such that $y^{(l)} = k$ holds. We have $\sum_{k=1}^K p_k = 1$

For an attribute $a \in A$, let $q_j(a)$ denotes the proportion of $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^n$ such that $x_a^{(l)} = j$ holds. We have $\sum_{j=1}^{K_a} q_j(a) = 1$.

The entropy of a node is $E(\mathcal{S}) = - \sum_{k=1}^K p_k \log_2 p_k$. If $p_k = 0$, we set $p_k \log_2 p_k = 0$.

The entropy after down-branching is $\sum_{j=1}^{K_a} q_j(a)E(\mathcal{S}_j)$, where $\mathcal{S}_j$ is the resulting child node with $x_a^{(l)} = j$.

We choose the best attribute $a^* = \arg_a \max_{a \in A'} \left(E(\mathcal{S}) - \sum_{j=1}^{K_a} q_j(a)E(\mathcal{S}_j) \right)$.

We can also use Gini index $G(\mathcal{S}) = 1 - \sum_{k=1}^K p_k^2$ as our impurity measure.

Overall algorithm:

```

Function dtree( $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^n, A', a_0$ )
  if  $\mathcal{S} = \emptyset$  then
    return  $a_0$ 
  else if  $\{y^{(l)}\}_{l=1}^n$  are identical then
    return  $y^{(1)}$ 
  else if  $A' = \emptyset$  then
    return majority vote of  $\{y^{(l)}\}_{l=1}^n$ .
  else
     $a^* = \arg_a \max_{a \in A'} \left( E(\mathcal{S}) - \sum_{j=1}^{K_a} q_j(a)E(\mathcal{S}_j) \right)$ 
    tree = a new tree with root  $a^*$ 
    for  $k \in \{1, \dots, K_{a^*}\}$ :
       $\mathcal{S}_k = \{(\mathbf{x}^{(l)}, y^{(l)}) \in \mathcal{S} : x_{a^*} = k\}$ 
       $a'_0 = \text{majority vote of } y^{(l)} \text{ in } \mathcal{S}_k$ 
      subtree = dtree( $\mathcal{S}_k, A' \setminus \{a^*\}, a'_0$ )
      add a branch  $a^* = k$  to tree with subtree subtree.
  return tree

```

Decision trees often leads to overfitting. To avoid this, we can use decision tree ensembles, i.e., employing multiple decision trees and use the average or majority voting of these results. We can use bagging or random forests to construct such decision trees.

Bagging (bootstrap aggregating): Bootstrap from $\mathcal{S} = \{(\mathbf{x}^{(l)}, y^{(l)})\}_{l=1}^N$ by drawing N examples randomly from $\mathcal{S}$ with replacement. Repeat the process multiple E times and we can get $\{\mathcal{S}_i\}_{i=1}^E$. We have $|\mathcal{S}_i| = |\mathcal{S}| = N \quad \forall i$

Random forest further modifies $a^* = \arg_a \max_{a \in \tilde{A}} \left(E(\mathcal{S}) - \sum_{j=1}^{K_a} q_j(a)E(\mathcal{S}_j) \right)$, where $\tilde{A} \subset A'$ is randomly sampled. e.g. $|\tilde{A}| \approx \sqrt{|A'|}$.